

Computer Organization & Assembly Language (EE2003)

Final Exam

Total Time (Hrs):

3

Total Marks:

60

Total Questions:

3

Date: December 20th 2025

Course Instructor(s)

Dr. Aatira Anum Ahmad, Ms. Samin Iftikhar

Ms. Aleena Ahmad, Ms. Sana Fatima

Mr. Zummar Saad

16.07 - 16.1
9.64 - 9.6
19.23 - 19.3

Roll No

Section

Student Signature

Do not write below this line

Instruction/Notes: This is an open book exam. Only textbook by "Belal Hashmi" is allowed. Sharing book or calculators is NOT ALLOWED. PAPERS/NOTES ATTACHED WITH BOOK ARE NOT ALLOWED. In case of any ambiguity, make reasonable assumptions. Questions during exams are not allowed. **ATTEMPT QUESTION 1 and 2 ON THIS QUESTION PAPER, SOLUTION ON ANSWER SHEET WILL NOT BE MARKED. Attempt Question 3 on answer sheet.**

CLO # 2: Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.

Q1: Short Questions [5+3+2+2+4+4+3+2 = 15 marks]

25 marks

(a) (i) What value is loaded into AX after executing MOV AX, [1234H] assuming little endian memory holds 78H at 1234H and 56H at 1235H?

a) 7856H b) 5678H c) 3478H d) 7812H

(ii) MOV AX, [BX] is an example of which addressing mode?

a) Register Indirect b) Immediate c) Register d) Direct

(iii) The instruction MOV AX, [bp+si] will by default use which segment?

a) CS b) DS c) SS d) ES

(iv) Je is a.....?

a) conditional jump b) short jump c) check zero flag d) All of the above

(v) If the IP = 1000H and CS = 2000H, the physical address of the next instruction is:

a) 21000H b) 30000H c) 12000H d) 20000H

1 mark each

National University of Computer and Emerging Sciences Lahore Campus

(b) What would be the value of SF and OF flag after execution of cmp instruction. Tell whether jump is taken or not?

```
Mov ax, -2
Mov bx, -1
cmp ax, bx
Jae exit
Add ax, 1
Exit
Mov ax, 0x4c00
Int 21h
```

SF: 1
OF: 0
Jump taken (Yes/No):
No

1 mark each

(c) What would be the value of bx register after execution of line #3 and after line #4?

```
1 [org 0x0100]
2 mov bx, 0x000F
3 test bx, 1
4 and bx, 1
5 Exit
6 Mov ax, 0x4c00
7 int 0x21
```

Line # 3: 0x000F
Line # 4: 0x0001

1 mark each

(d) Following program is trying to make the font color white without changing anything else on the screen. Write mask operation to complete this task.

```
[org 0x0100]
mov ax, 0xb800
mov es, ax
mov di, 0
nextchar: mov ax, word [es:di]
```

OR AH, 0x07; write mask operation here

```
mov word [es:di], ax
add di, 2
cmp di, 4000
jne nextchar
mov ax, 0x4c00
int 0x21
```

2 OR new

(e) Initial values of registers are as follows: AX= 1111h BX= 2222h, CX=5555h, DX= 8888h. What will be the value of registers after the execution of the following code?

```
F1: Pop ax
Push cx
Pop dx
Push ax
Ret
Start:
call F1
Mov ax, 7777h
Mov bx, 3333h
```

AX: 7777
BX: 3333
CX: 5555
DX: 5555

1 mark each

(f) Offset of start is 0x0253. If we perform the following code, how many paragraphs will be saved?

```
mov dx, start
mov cl, 4
shr dx, cl
mov ax, 0x3100
int 0x21
```

No of Paragraphs: (37)₁₀

0x25

4 OR new

(g) What will be the values of SI, DI and CX after the execution of the following code? The addresses of str1 and str2 are 0103h and 010Dh.

```
[org 0x100]
jmp start
str1 db 'ABCDEFGH', 0
str2 db 'ABCDEFGHIJKLM', 0
start:
PUSH DS
POP ES
MOV SI, str1
MOV DI, str2
MOV CX, 15
CLD
REPE CMPSB
```

SI: 010D
DI: 0117
CX: 05

1 mark each

(h) What logical address (segment: offset pair) line "mov ax, [bx]" is reading if cs = ds = ss = es = 0x19F5.

```
[org 0x0100]
2 00000000 8B1E[1100]      mov bx, [num1+8]
3 00000004 8B07            mov ax, [bx]
4 00000006 B8004C          mov ax, 0x4C00, terminate program
5 00000009 CD21            int 0x21
6 0000000B 010002000300040005 num1 dw 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
6 00000014 000600070008000900
6 0000001D 0A00
```

Segment: 19F5
Offset: 0005

CLO # 3: Apply the knowledge of

Intel x86 architecture to develop moderately complex and well-modularized assembly programs.

Q2: [5x3 = 15 Marks]: An Elaborate Multitasking example 10.2 is provided in the book. Make the following changes to the example. In your solution only following is required:

Lines of code which need to be removed

Lines of code which need to be modified along with modification

Lines of code which need to be added.

No credit will be given for anything else.

a) The program supports only 8 tasks with 128 words stack size.

① task: times 8x16 dw 0
② stack: " 8x128 dw 0

initpcb:
cmp bx, 8 ①
mov cl, 8 ②
shr si, cl
add si, 128x2 + stack

b) Currently the example handles multiple threads of number printing at the 70th column of multiple rows and myTask takes only one parameter i.e., row number. Modify the program such that every task prints the number at row and column both given in parameters. Now myTask has 2 parameters; row number and column number. This column number is passed as a parameter to initpcb subroutine just like lineno. You can initialize the column number with any number.

initpcb:
102. mov ax, [bp+10] ; segment ①
104. mov ax, [bp+8] ; offset ①
mov ax, [bp+6] ; parameter
sub si, 2
mov [si], ax ; param on stack ②
ret 8 ①

c) In the original example, a new task is initialized when any key is pressed. Modify the code so that after escape key (ASCII: 0x1B) is pressed, no more tasks will be initialized.

after int 0x16

if ~~esc~~ escape ; ~~if~~ continue without hit pcb
~~else~~ jmp \$; call
else int pcb call code

CLO # 3: Apply the knowledge of Intel x86 architecture to develop moderately complex and well-modularized assembly programs.

Q3: [30 Marks] In this question, you will write code to implement a shootDigits game (inspired from shootBubbles game). The game starts with random digits from 0-9 placed at the top 5 rows of the display memory. These digits will move down one row automatically after some time. A white box is initially placed in the center of the last row which can be moved left and right using arrow keys. If the user press enter key, the last digit placed in the column where the box is currently placed is replaced with space along with the surrounding cells with the same digit. The user wins the game if all the digits are converted to space before the digits touch the last row of the display screen. The user loses if the digits touch the last row.

You have to implement the following parts of the game.

- (1) **Initial Screen**
 - a. Clear the screen by calling clrscr subroutine. [2]
 - b. Call placeDigits subroutine that will randomly place digits in the 1st 5 rows of the display screen. You do not need to implement this subroutine. [2]
 - c. Place a white box (ASCII: 0xDB) in the 39th column of the last row. [2]
- (2) **Game Mechanism**
 - a. The box will move one step to left and right using the left and right arrow keys respectively. (scan code of left arrow key: 0x4B, scan code of right arrow key: 0x4D) Do not need to add boundary checks. [4]
 - b. When user presses the enter key (scan code: 0x1C), the last digit in the column where the box is currently placed is converted to space along with the surrounding cells with the same digit. The surrounding cells include above, left, and right cells only as shown in the figures below. You do not need to add boundary checks. [6]
 - c. After every 20 timer interrupts, the digits will move downward one row clearing the above row unless it touches the last row. [6]
 - d. If the digits touch the last row, the game will be over. You do not need to implement the winning condition. [6]
 - e. After the end of the game, clear the screen, safely exit the program (unhook keyboard and timer interrupts) so that other programs can run smoothly on the DOSBox. [2]

In the figure below on the left, the white box is placed at column 3 and the last digit in this column is 9. So, when Enter key is pressed, this 9 will be converted in to space. As the left and right digit is also 9, they will also be converted to space as shown in figure below on the right but the above cell will remain same as the number is 3 not 9

Important Instructions:

- If you want to use any function from textbook, simply call it PROPERLY. YOU DO NOT NEED TO RE-WRITE IT.
- If you want to use the code to hook or unhook Keyboard or Timer interrupt Simply write:
 "Code to hook Timer comes here"
 "Code to unhook Timer comes here"
 You do not need to re-write the code but do clearly mention this statement.
- Properly comment your code and write functions where needed.
- Freely use global variables, if required.

1	9	7	6	3	1	9	7	6	3
3	5	3	3	3	3	5	3	3	3
2	2	2	7	8	2	2	2	7	8
9	7	3	2	1	9	7	3	2	1
1	9	9	9	4	1				4
Figure before pressing Enter key					Figure after pressing Enter key				